Tarefa2: Lab03Tarefa04 – Agrupamento Hierárquico

Nome: Leonardo Scholl Sternberg

Curso: Algoritmos para Agrupamento de Dados

Especialização em Inteligência Artificial para Engenheiros

Avaliação do Código

1: Configuração, Carga e Preparação dos Dados

```python
# Importa as bibliotecas necessárias
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
%matplotlib inline
import numpy as np
from scipy.cluster.hierarchy import dendrogram
from scipy.cluster.hierarchy import linkage
```

✓ 0.0s

```python
dados = pd.read_csv('./DadosWH.csv')
MB = dados.year == 2019
dados2019 = dados[MB]
dados2019
```

✓ 0.0s

	Name	Continent	year	population	Life_Ladder	Log_GDP_per_capita	Social_support	Healthy_life_expectancy_at_birth	Freedom_to_make_life_choices	Generosity	Perceptions_of_corruption	Positive_affect	Negative_affect
9	Afghanistan	Asia	2019	38041754.0	2.375	7.697	0.420	52.4	0.394	-0.108	0.924	0.351	0.502
19	Albania	Europe	2019	2854191.0	4.995	9.544	0.686	69.0	0.777	-0.099	0.914	0.681	0.274
29	Algeria	Africa	2019	43053054.0	4.745	9.337	0.803	66.1	0.385	0.005	0.741	0.585	0.215
39	Argentina	South America	2019	44938712.0	6.086	10.000	0.896	69.0	0.817	-0.211	0.830	0.826	0.319
49	Armenia	Europe	2019	2957731.0	5.488	9.522	0.782	67.2	0.844	-0.172	0.583	0.598	0.430
...	...	...	...	...	...	...	...	...	...	...	...	...	...
1179	Uruguay	South America	2019	3461734.0	6.600	9.979	0.933	69.1	0.903	-0.095	0.599	0.889	0.222
1189	Uzbekistan	Asia	2019	33580650.0	6.154	8.853	0.915	65.4	0.970	0.304	0.511	0.845	0.220
1199	Vietnam	Asia	2019	96462106.0	5.467	8.992	0.848	68.1	0.952	-0.126	0.788	0.751	0.186
1209	Zambia	Africa	2019	17861030.0	3.307	8.155	0.638	55.8	0.811	0.077	0.832	0.743	0.394
1219	Zimbabwe	Africa	2019	14645468.0	2.694	7.950	0.759	56.2	0.632	-0.064	0.831	0.716	0.235

22 rows × 13 columns

```python
# Normaliza os dados das caracteristicas consideradas (Life_Ladder e Perceptions_of_corruption)
dimensoes = ['Life_Ladder','Perceptions_of_corruption']
Xs = dados2019[dimensoes]
```

✓ 0.0s

```python
Xs = (Xs - Xs.min())/(Xs.max()-Xs.min())
Xs.index = dados2019['Name']
Xs.head()
```

✓ 0.0s

Name	Life_Ladder	Perceptions_of_corruption
Afghanistan	0.000000	0.956327
Albania	0.484736	0.945129
Algeria	0.438483	0.751400
Argentina	0.686586	0.851064
Armenia	0.575948	0.574468

Este bloco inicial estabelece o ambiente de trabalho e realiza o pré-processamento necessário para a análise de agrupamento:

- **Bibliotecas e Ferramentas:** Importação do Pandas e NumPy para manipulação de dados, além do Matplotlib e SciPy para o processamento e visualização hierárquica.
- **Filtragem Temporal:** Carga do dataset “DadosWH.csv” e aplicação de filtro para o ano de “2019”.

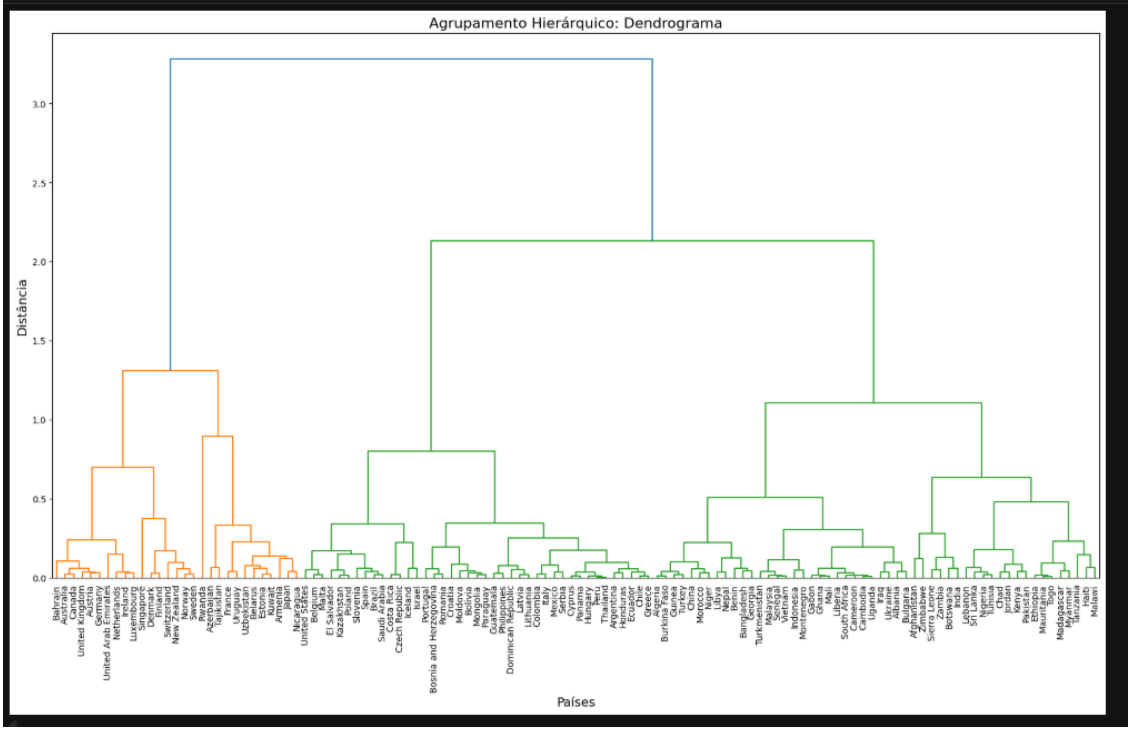
- **Seleção de Atributos:** Extração das colunas “Life_Ladder” e “Perceptions_of_corruption” como variáveis base para o cálculo de similaridade entre os países.
- **Normalização e Indexação:** Aplicação da escala Min-Max (intervalo 0 a 1) para padronizar as grandezas e atribuição da coluna “Name” como índice, permitindo a identificação dos países no dendrograma.

2: Comparação das Estruturas Hierárquicas

Este bloco realiza o processamento do agrupamento hierárquico e gera os dendrogramas correspondentes, permitindo comparar como cada critério de ligação organiza a hierarquia entre os países:

Método de Ward:

```
# Plota o dendrograma considerando o método de Ward
plt.figure(figsize=(21,11))
plt.title('Agrupamento Hierárquico: Dendrograma', fontsize=16)
dendrogram_plot = dendrogram(linkage(Xs, method = 'ward'), labels=Xs.index)
plt.xlabel('Países', fontsize=14)
plt.ylabel('Distância', fontsize=14)
plt.tick_params(axis='x', labelsize=10) # Adjust x-axis tick label size
plt.tick_params(axis='y', labelsize=10) # Adjust y-axis tick label size
plt.show()
✓ 0.3s
```

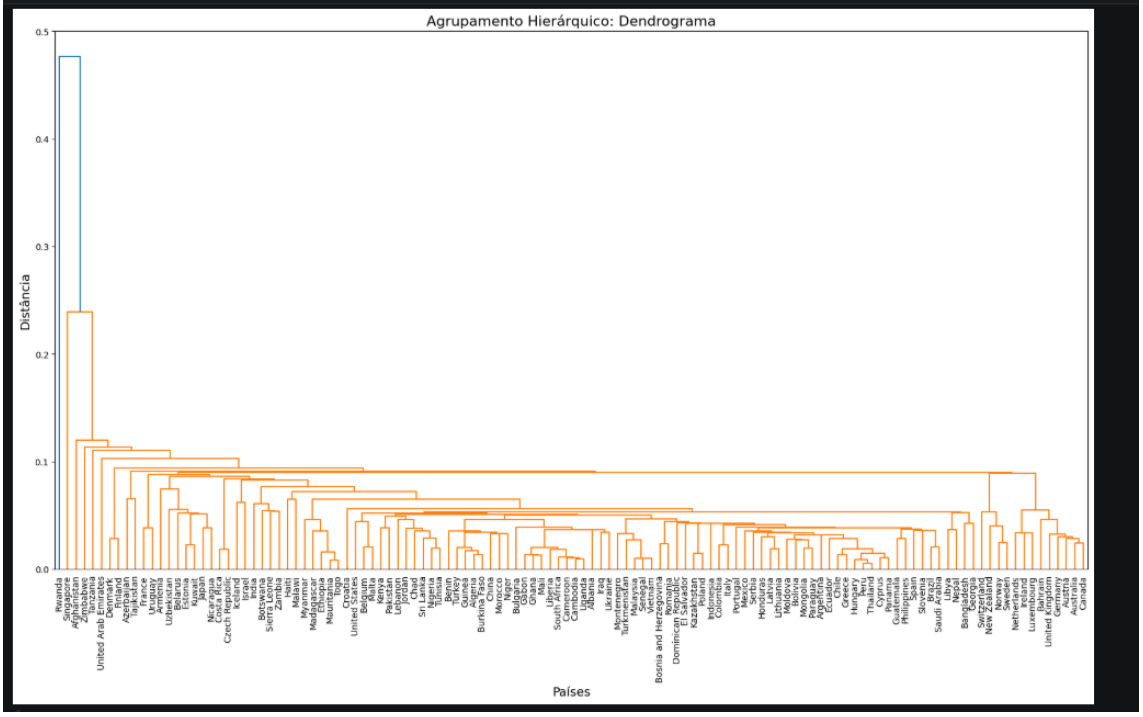


- **Lógica:** Minimiza a variância interna ao fundir clusters, buscando grupos o mais homogêneos possível.
- **Visual do Dendrograma:** presenta as divisões mais claras e equilibradas. As linhas verticais são longas, o que facilita a visualização de grupos distintos e a definição de um ponto de corte.
- **Aplicação:** É o método mais indicado quando se busca uma partição bem definida, equilibrada e com alta coesão interna.

Método Single (Vizinho Mais Próximo):

```python
# Plota o dendrograma considerando o método de vizinho mais próximo
plt.figure(figsize=(21,11))
plt.title('Agrupamento Hierárquico: Dendrograma', fontsize=16)
dendrogram_plot = dendrogram(linkage(Xs, method = 'single'), labels=Xs.index)
plt.xlabel('Países', fontsize=14)
plt.ylabel('Distância', fontsize=14)
plt.tick_params(axis='x', labelsize=10) # Adjust x-axis tick label size
plt.tick_params(axis='y', labelsize=10) # Adjust y-axis tick label size
plt.show()
```

✓ 0.3s

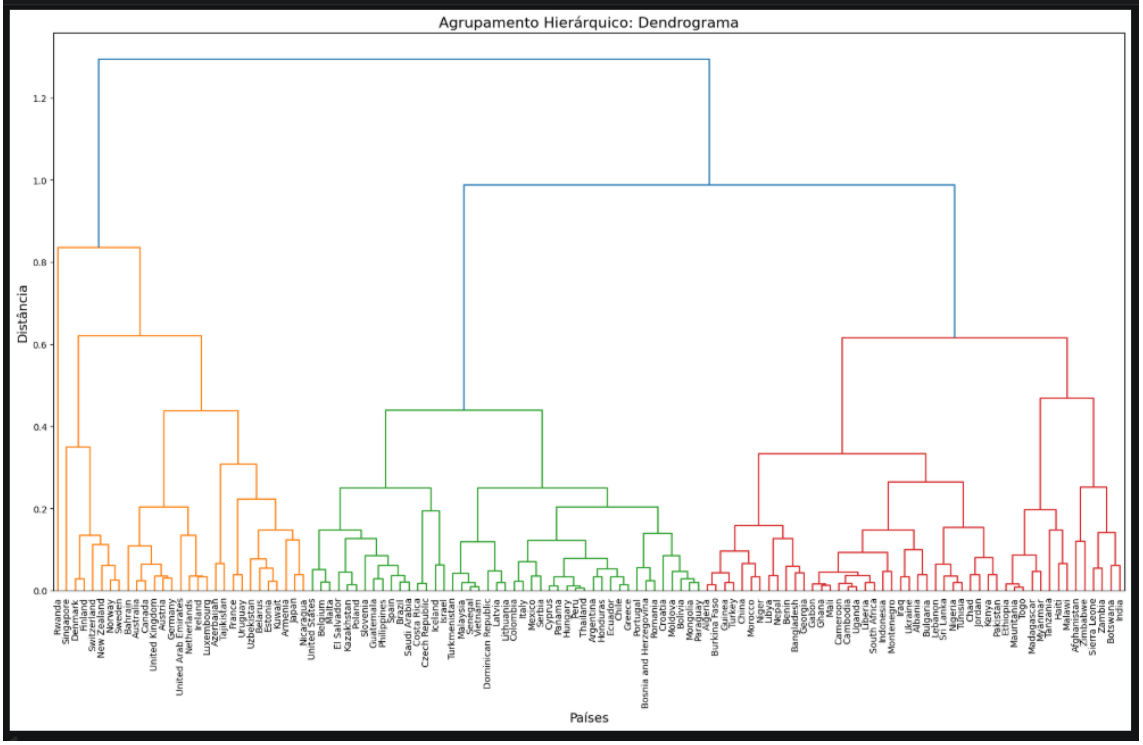


- **Lógica:** Baseia-se na distância mínima entre os membros mais próximos de dois grupos.
- **Visual do Dendrograma:** Exibe o efeito de **encadeamento** (*chaining*), onde os países são adicionados individualmente à estrutura, criando uma aparência de escada.
- **Aplicação:** É o método menos indicado para criar partições homogêneas, sendo mais útil para a identificação de *outliers* ou cadeias de similaridade.

Método Complete (Vizinho Mais Distante):

```python
# Plota o dendrograma considerando o método do vizinho mais distante
plt.figure(figsize=(21,11))
plt.title('Agrupamento Hierárquico: Dendrograma', fontsize=16)
dendrogram_plot = dendrogram(linkage(Xs, method = 'complete'), labels=Xs.index)
plt.xlabel('Países', fontsize=14)
plt.ylabel('Distância', fontsize=14)
plt.tick_params(axis='x', labelsize=10) # Adjust x-axis tick label size
plt.tick_params(axis='y', labelsize=10) # Adjust y-axis tick label size
plt.show()
```

✓ 0.2s



- **Lógica:** Considera a maior distância entre os membros para realizar a fusão.
- **Visual do Dendrograma:** Apresenta uma estrutura intermediária entre o Ward e o Single.
- **Aplicação:** Quando se deseja garantir que os elementos dentro de um mesmo grupo não sejam excessivamente diferentes entre si em nenhuma das dimensões.

3: Reestruturação e Conferência do DataFrame

Qual o número de clusters que vocês considerariam o ideal?

```
Xs.index = dados2019.index
Xs['Name'] = dados2019['Name']
Xs.head()
```

✓ 0.0s

	Life_Ladder	Perceptions_of_corruption	Name
9	0.000000	0.956327	Afghanistan
19	0.484736	0.945129	Albania
29	0.438483	0.751400	Algeria
39	0.686586	0.851064	Argentina
49	0.575948	0.574468	Armenia

Este bloco realiza ajustes nos índices e metadados do DataFrame processado para garantir a rastreabilidade dos dados:

- **Sincronização de Índices:** O comando Xs.index = dados2019.index garante que o DataFrame normalizado mantenha os mesmos identificadores numéricos (IDs) da base original.
- **Reinserção de Identificadores:** A criação da coluna Xs['Name'] traz de volta os nomes dos países para dentro do conjunto de dados normalizados, facilitando a conferência visual.
- **Inspeção de Dados (head):** Exibição das primeiras linhas para validar se a normalização (valores entre 0 e 1) e a atribuição dos nomes foram executadas corretamente antes das etapas seguintes.

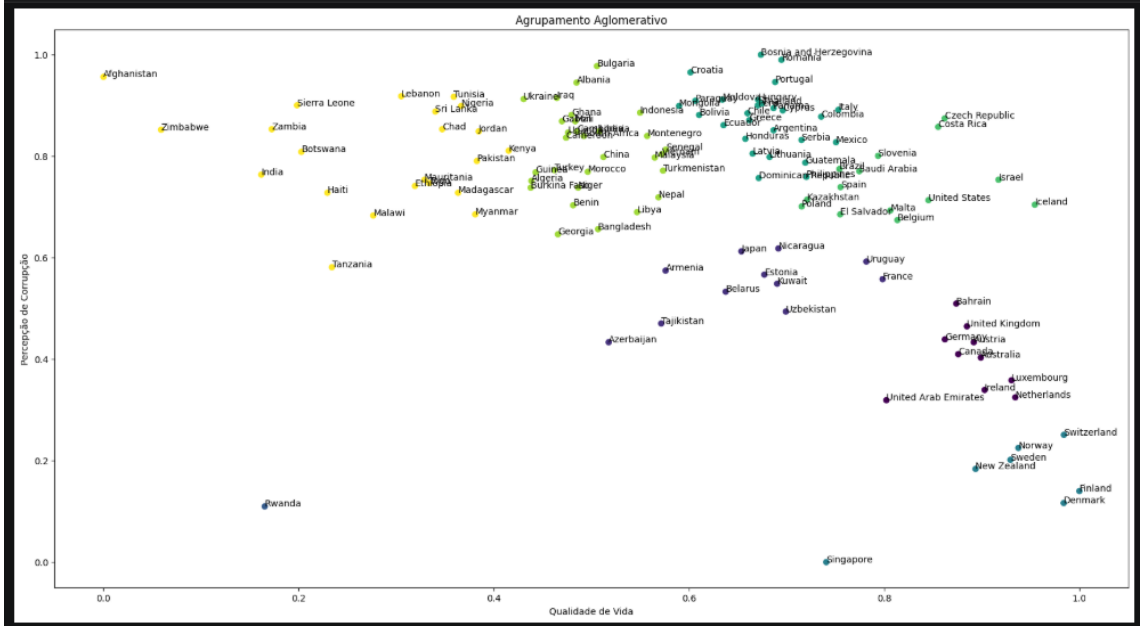
4: Comparação dos Métodos de Agrupamento Aglomerativo

Este bloco utiliza o algoritmo AgglomerativeClustering para testar diferentes critérios de ligação, classificando os países em 8 clusters distintos e mapeando os resultados em gráficos de dispersão:

Método de Ward (Minimização de Variância):

```python
# Aplica o método de Ward para o agrupamento hierárquico
from sklearn.cluster import AgglomerativeClustering
num_clusters = 8
ac_ward = AgglomerativeClustering(n_clusters = num_clusters, linkage='ward')
ac_ward.fit(Xs)
pred_ward=ac_ward.labels_
```

```python
# Plota as amostras com a identificação do cluster do Ward
plt.figure(figsize=(21,11))
plt.scatter(Xs.Life_Ladder,Xs.Perceptions_of_corruption,c=pred_ward, marker="o", cmap='viridis_r')
for _, x in Xs.iterrows():
    plt.annotate(x.Name, (x.Life_Ladder, x.Perceptions_of_corruption))
plt.title('Agrupamento Aglomerativo')
plt.xlabel('Qualidade de Vida')
plt.ylabel('Percepção de Corrupção')
plt.show()
```

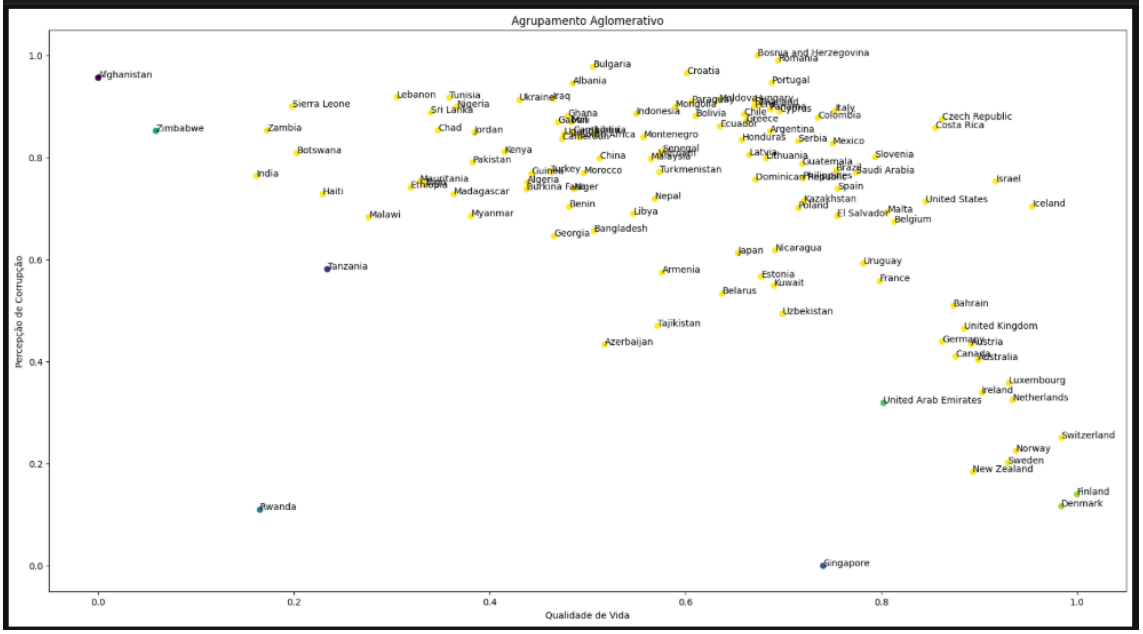


- **Comportamento:** Focado em reduzir a soma dos quadrados das distâncias dentro de cada grupo.
- **Resultado Visual:** É o que gera a distribuição mais equilibrada no gráfico. Os clusters aparecem como blocos compactos e bem distribuídos, facilitando a interpretação de categorias socioeconômicas claras (ex: países com alta qualidade de vida e baixa corrupção agrupados em uma extremidade).

Método Single (Vizinho Mais Próximo):

```python
# Aplica o método de vizinho mais próximo para o agrupamento hierárquico
from sklearn.cluster import AgglomerativeClustering
num_clusters = 8
ac_single = AgglomerativeClustering(n_clusters = num_clusters, linkage='single')
ac_single.fit(Xs)
pred_single=ac_single.labels_
```

```python
# Plota as amostras com a identificação do cluster do vizinho mais próximo
plt.figure(figsize=(21,11))
plt.scatter(Xs.Life_Ladder,Xs.Perceptions_of_corruption,c=pred_single, marker="o", cmap='viridis_r')
for _, x in Xs.iterrows():
    plt.annotate(x.Name, (x.Life_Ladder, x.Perceptions_of_corruption))
plt.title('Agrupamento Aglomerativo')
plt.xlabel('Qualidade de Vida')
plt.ylabel('Percepção de Corrupção')
plt.show()
```

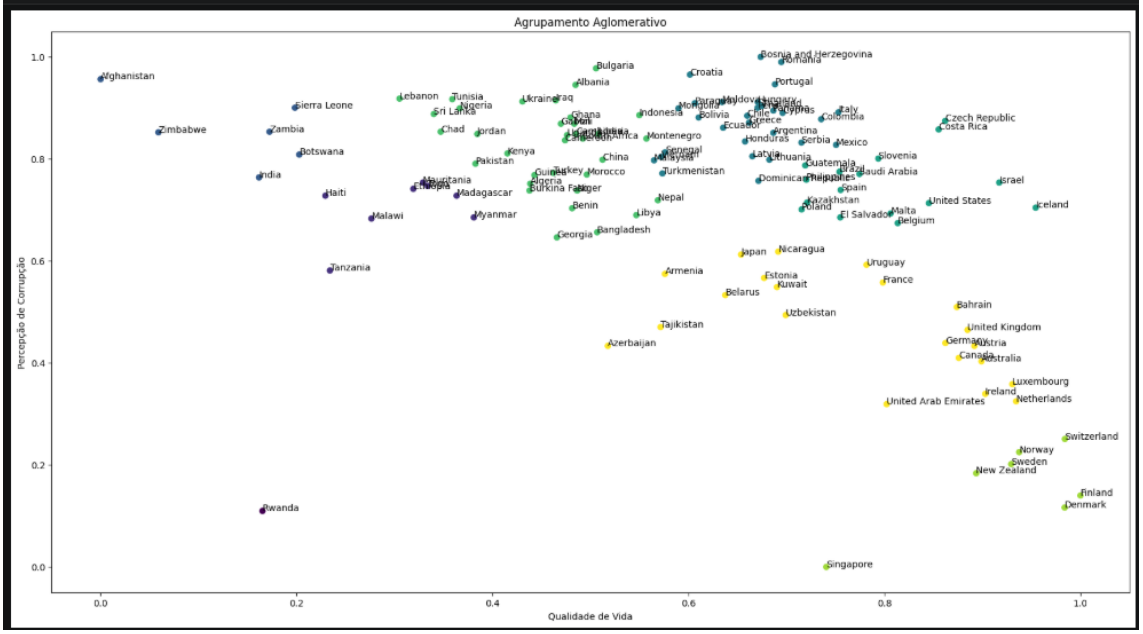


- **Comportamento:** Agrupa elementos com base na menor distância entre qualquer par de pontos.
- **Resultado Visual:** Evidencia o fenômeno de encadeamento (*chaining*). No gráfico, isso resulta em um grande cluster central que engloba quase todos os países, enquanto pontos isolados ou extremos (outliers), como Singapura ou Ruanda, acabam formando clusters solitários.

Método Complete (Vizinho Mais Distante):

```python
# Aplica o método do vizinho mais distante para o agrupamento hierárquico
from sklearn.cluster import AgglomerativeClustering
num_clusters = 8
ac_complete = AgglomerativeClustering(n_clusters = num_clusters, linkage='complete')
ac_complete.fit(Xs)
pred_complete=ac_complete.labels_
```

```python
# Plota as amostras com a identificação do cluster do vizinho mais distante
plt.figure(figsize=(21,11))
plt.scatter(Xs.Life_Ladder,Xs.Perceptions_of_corruption,c=pred_complete, marker="o", cmap='viridis_r')
for _, x in Xs.iterrows():
    plt.annotate(x.Name, (x.Life_Ladder, x.Perceptions_of_corruption))
plt.title('Agrupamento Aglomerativo')
plt.xlabel('Qualidade de Vida')
plt.ylabel('Percepção de Corrupção')
plt.show()
```



- **Comportamento**: Define a distância entre clusters pelo par de elementos mais afastado, impondo um limite máximo de "diâmetro" para cada grupo.
- **Resultado Visual:** Produz uma segmentação mais fragmentada que o Single, mas com fronteiras mais rígidas que o Ward. No plano bidimensional, os grupos aparecem como divisões espaciais mais lineares, sendo útil para garantir que nenhum membro de um cluster seja excessivamente diferente dos demais.

5: Processamento das Matrizes de Ligação

Este bloco realiza o processamento central do agrupamento hierárquico, gerando as matrizes de ligação que armazenam matematicamente como os dados são fundidos:

```python
# Aplica o método de Mojena para a determinação do número de clusters
from scipy.cluster.hierarchy import linkage, dendrogram
from sklearn.preprocessing import StandardScaler

# Remove a coluna 'Name' para o agrupamento, se ela existir
Xs_numeric = Xs.drop('Name', axis=1, errors='ignore')

# Aplica o método de Ward para o agrupamento hierárquico do Ward
Z_ward = linkage(Xs_numeric, method='ward')
Z_ward
```

✓ 0.0s

```
array([[8.80000000e+01, 1.07000000e+02, 5.42159840e-03, 2.00000000e+00],
       [7.10000000e+01, 1.08000000e+02, 8.26338451e-03, 2.00000000e+00],
       [1.90000000e+01, 1.12000000e+02, 9.51794781e-03, 2.00000000e+00],
       [9.50000000e+01, 1.19000000e+02, 9.83988318e-03, 2.00000000e+00],
       [2.80000000e+01, 8.60000000e+01, 1.04449298e-02, 2.00000000e+00],
       [4.70000000e+01, 1.22000000e+02, 1.12175623e-02, 3.00000000e+00],
       [4.10000000e+01, 6.90000000e+01, 1.29162429e-02, 2.00000000e+00],
       [2.00000000e+00, 1.80000000e+01, 1.34582129e-02, 2.00000000e+00],
       [2.30000000e+01, 4.20000000e+01, 1.35646166e-02, 2.00000000e+00],
       [5.70000000e+01, 9.00000000e+01, 1.45390141e-02, 2.00000000e+00],
       [2.00000000e+01, 1.24000000e+02, 1.50449160e-02, 3.00000000e+00],
       [3.80000000e+01, 1.28000000e+02, 1.53170023e-02, 3.00000000e+00],
       [6.80000000e+01, 1.25000000e+02, 1.67720086e-02, 3.00000000e+00],
       [3.50000000e+01, 1.23000000e+02, 1.74876101e-02, 3.00000000e+00],
       [1.00000000e+02, 1.32000000e+02, 1.75058646e-02, 4.00000000e+00],
       [2.60000000e+01, 2.90000000e+01, 1.80696419e-02, 2.00000000e+00],
       [6.00000000e+01, 6.40000000e+01, 1.86440720e-02, 2.00000000e+00],
       [8.30000000e+01, 1.09000000e+02, 1.94567346e-02, 2.00000000e+00],
       [7.40000000e+01, 8.70000000e+01, 1.94637607e-02, 2.00000000e+00],
       [4.40000000e+01, 1.10000000e+02, 1.97559386e-02, 2.00000000e+00],
       [1.10000000e+01, 7.00000000e+01, 2.03585278e-02, 2.00000000e+00],
       [1.60000000e+01, 9.40000000e+01, 2.08386325e-02, 2.00000000e+00],
       [2.50000000e+01, 5.40000000e+01, 2.21247096e-02, 2.00000000e+00],
       [3.40000000e+01, 5.90000000e+01, 2.22166103e-02, 2.00000000e+00],
       [1.40000000e+01, 9.20000000e+01, 2.33758148e-02, 2.00000000e+00],
...
       [9.30000000e+01, 2.29000000e+02, 8.95960343e-01, 1.20000000e+01],
       [2.34000000e+02, 2.35000000e+02, 1.10718298e+00, 5.20000000e+01],
       [2.36000000e+02, 2.38000000e+02, 1.30830315e+00, 2.90000000e+01],
       [2.37000000e+02, 2.39000000e+02, 2.12989171e+00, 9.30000000e+01],
       [2.40000000e+02, 2.41000000e+02, 3.28019056e+00, 1.22000000e+02]])
```

```python
# Aplica o método de Ward para o agrupamento hierárquico do vizinho mais próximo
Z_single = linkage(Xs_numeric, method='single')
Z_single
```

✓ 0.0s

```
rray([[8.80000000e+01, 1.07000000e+02, 5.42159840e-03, 2.00000000e+00],
       [7.10000000e+01, 1.08000000e+02, 8.26338451e-03, 2.00000000e+00],
       [4.70000000e+01, 1.22000000e+02, 8.83251497e-03, 3.00000000e+00],
       [1.90000000e+01, 1.12000000e+02, 9.51794781e-03, 2.00000000e+00],
       [9.50000000e+01, 1.19000000e+02, 9.83988318e-03, 2.00000000e+00],
       [6.80000000e+01, 1.26000000e+02, 9.85941109e-03, 3.00000000e+00],
       [2.00000000e+01, 1.25000000e+02, 1.02818160e-02, 3.00000000e+00],
       [2.80000000e+01, 8.60000000e+01, 1.04449298e-02, 2.00000000e+00],
       [1.00000000e+02, 1.28000000e+02, 1.12746162e-02, 4.00000000e+00],
       [4.10000000e+01, 6.90000000e+01, 1.29162429e-02, 2.00000000e+00],
       [2.00000000e+00, 1.80000000e+01, 1.34582129e-02, 2.00000000e+00],
       [2.30000000e+01, 4.20000000e+01, 1.35646166e-02, 2.00000000e+00],
       [3.80000000e+01, 1.31000000e+02, 1.36910268e-02, 3.00000000e+00],
       [5.70000000e+01, 9.00000000e+01, 1.45390141e-02, 2.00000000e+00],
       [1.24000000e+02, 1.29000000e+02, 1.48479812e-02, 5.00000000e+00],
       [3.50000000e+01, 1.23000000e+02, 1.56518445e-02, 3.00000000e+00],
       [4.40000000e+01, 1.32000000e+02, 1.73279383e-02, 3.00000000e+00],
       [2.60000000e+01, 2.90000000e+01, 1.80696419e-02, 2.00000000e+00],
       [6.20000000e+01, 1.30000000e+02, 1.82598464e-02, 5.00000000e+00],
       [6.00000000e+01, 6.40000000e+01, 1.86440720e-02, 2.00000000e+00],
       [1.33000000e+02, 1.36000000e+02, 1.88943761e-02, 7.00000000e+00],
       [8.30000000e+01, 1.09000000e+02, 1.94567346e-02, 2.00000000e+00],
       [7.40000000e+01, 8.70000000e+01, 1.94637607e-02, 2.00000000e+00],
       [1.10000000e+02, 1.38000000e+02, 1.97559386e-02, 4.00000000e+00],
       [1.34000000e+02, 1.40000000e+02, 2.02415066e-02, 8.00000000e+00],
..
       [1.06000000e+02, 2.37000000e+02, 1.10360435e-01, 1.18000000e+02],
       [1.21000000e+02, 2.38000000e+02, 1.13419034e-01, 1.19000000e+02],
       [0.00000000e+00, 2.39000000e+02, 1.19704333e-01, 1.20000000e+02],
       [9.80000000e+01, 2.40000000e+02, 2.39036671e-01, 1.21000000e+02],
       [9.30000000e+01, 2.41000000e+02, 4.76441900e-01, 1.22000000e+02]])
```

```python
# Aplica o método de Ward para o agrupamento hierárquico do vizinho mais distante
Z_complete = linkage(Xs_numeric, method='complete')
Z_complete
```

✓ 0.0s

```
array([[8.80000000e+01, 1.07000000e+02, 5.42159840e-03, 2.00000000e+00],
       [7.10000000e+01, 1.08000000e+02, 8.26338451e-03, 2.00000000e+00],
       [1.90000000e+01, 1.12000000e+02, 9.51794781e-03, 2.00000000e+00],
       [9.50000000e+01, 1.19000000e+02, 9.83988318e-03, 2.00000000e+00],
       [2.80000000e+01, 8.60000000e+01, 1.04449298e-02, 2.00000000e+00],
       [4.70000000e+01, 1.22000000e+02, 1.11997366e-02, 3.00000000e+00],
       [4.10000000e+01, 6.90000000e+01, 1.29162429e-02, 2.00000000e+00],
       [2.00000000e+00, 1.80000000e+01, 1.34582129e-02, 2.00000000e+00],
       [2.30000000e+01, 4.20000000e+01, 1.35646166e-02, 2.00000000e+00],
       [5.70000000e+01, 9.00000000e+01, 1.45390141e-02, 2.00000000e+00],
       [3.80000000e+01, 1.28000000e+02, 1.57444050e-02, 3.00000000e+00],
       [3.50000000e+01, 1.23000000e+02, 1.57444050e-02, 3.00000000e+00],
       [2.00000000e+01, 1.24000000e+02, 1.67064111e-02, 3.00000000e+00],
       [2.60000000e+01, 2.90000000e+01, 1.80696419e-02, 2.00000000e+00],
       [6.20000000e+01, 1.00000000e+02, 1.82598464e-02, 2.00000000e+00],
       [6.00000000e+01, 6.40000000e+01, 1.86440720e-02, 2.00000000e+00],
       [6.80000000e+01, 1.25000000e+02, 1.93171963e-02, 3.00000000e+00],
       [8.30000000e+01, 1.09000000e+02, 1.94567346e-02, 2.00000000e+00],
       [7.40000000e+01, 8.70000000e+01, 1.94637607e-02, 2.00000000e+00],
       [4.40000000e+01, 1.10000000e+02, 1.97559386e-02, 2.00000000e+00],
       [1.10000000e+01, 7.00000000e+01, 2.03585278e-02, 2.00000000e+00],
       [1.60000000e+01, 9.40000000e+01, 2.08386325e-02, 2.00000000e+00],
       [2.50000000e+01, 5.40000000e+01, 2.21247096e-02, 2.00000000e+00],
       [3.40000000e+01, 5.90000000e+01, 2.22166103e-02, 2.00000000e+00],
       [1.40000000e+01, 9.20000000e+01, 2.33758148e-02, 2.00000000e+00],
...
       [2.33000000e+02, 2.37000000e+02, 6.16292585e-01, 4.80000000e+01],
       [2.34000000e+02, 2.35000000e+02, 6.20082436e-01, 2.80000000e+01],
       [9.30000000e+01, 2.39000000e+02, 8.35329976e-01, 2.90000000e+01],
       [2.36000000e+02, 2.38000000e+02, 9.87003128e-01, 9.30000000e+01],
       [2.40000000e+02, 2.41000000e+02, 1.29361545e+00, 1.22000000e+02]])
```

- **Preparação dos Dados (Xs_numeric):** O código garante que apenas colunas quantitativas sejam utilizadas para os cálculos de distância, removendo a coluna 'Name' para evitar erros de processamento e assegurando que a matriz resultante seja puramente numérica.
- **Matriz de Ward (Z_ward):** Utiliza o método de Ward para calcular as fusões. Matematicamente, as distâncias exibidas na matriz Z_ward (terceira coluna do array) tendem a ser maiores, pois representam o aumento da variância total do sistema a cada união.
- **Matriz Single (Z_single):** Calcula as distâncias baseadas no critério do vizinho mais próximo. Nos arrays exibidos, é possível notar que os valores de distância crescem de forma muito mais lenta, refletindo fusões baseadas em pontos mínimos de contato.
- **Matriz Complete (Z_complete):** Calcula as fusões com base na maior distância entre os membros dos clusters. A matriz resultante apresenta valores de distância intermediários, garantindo que nenhum cluster exceda um determinado "diâmetro" matemático.
- **Estrutura das Matrizes:** Os resultados impressos em tela mostram o formato padrão do SciPy:
    - As duas primeiras colunas indicam os índices dos clusters/pontos sendo unidos.
    - A terceira coluna indica a **distância** calculada entre eles.
    - A quarta coluna indica o **número total de observações** contidas no novo cluster formado.

6: Implementação do Método de Mojena

```python
# 3. Implementação do Método de Mojena
def mojena_fator(Z):
    """
    Implementa o método de Mojena para agrupamento hierárquico.
    Z: Matriz linkage do scipy
    k_factor: Valor padrão geralmente entre 1.25 e 1.50
    """
    # Distâncias de fusão (terceira coluna da matriz Z)
    distances = Z[:, 2]

    # Calcular média e desvio padrão das distâncias
    mean_dist = np.mean(distances[0:-1])
    std_dist = np.std(distances[0:-1])

    # Calcular critério de Mojena (k) para cada estágio
    # k_j = (alpha_j - mean) / std
    mojena_k = (distances[-1] - mean_dist) / std_dist

    return mojena_k
```

Este bloco define uma função customizada para aplicar o **Método de Mojena**, um critério estatístico utilizado para determinar o número ideal de clusters em um agrupamento hierárquico, indo além da análise puramente visual do dendrograma:

- **Lógica do Algoritmo:** A função mojena_fator(Z) utiliza a matriz de ligação Z para calcular a média e o desvio padrão das distâncias de fusão de todos os estágios do agrupamento, exceto o último.
- **Cálculo do Critério:** O método baseia-se na fórmula:

$$k = \frac{(d_j - \overline{d})}{\sigma_d}$$

Onde d_j é a distância de fusão no estágio atual, $\overline{d}$ é a média das distâncias e σ_d é o desvio padrão.

- **Tomada de Decisão:** O valor retornado (mojena_k) ajuda a identificar o ponto de "salto" nas distâncias de fusão. Geralmente, interrompe-se o agrupamento quando o valor de k excede um patamar pré-definido (comumente entre 1.25 e 1.50), indicando que a próxima união fundiria grupos significativamente diferentes.
- **Automação da Análise:** Ao implementar essa função, o código prepara o terreno para uma escolha de número de clusters baseada em evidências estatísticas, permitindo comparar se o corte sugerido pela matemática coincide com a percepção visual obtida nos dendrogramas anteriores.

7: Aplicação do Método de Mojena e Análise do Fator por Cluster

Este bloco executa o cálculo iterativo do Fator de Mojena para as três matrizes de ligação (**Ward**, **Single** e **Complete**), gerando gráficos que correlacionam o valor do fator com a quantidade de clusters resultantes:

- **Loop de Cálculo:** O código percorre diferentes quantidades de clusters (de 3 até o limite total da amostra), aplicando a função mojena_fator em subconjuntos crescentes das matrizes de ligação.
- **Inversão de Dados (reverse):** Como o agrupamento hierárquico é aglomerativo (parte de muitos clusters para um só), os resultados são invertidos para que o gráfico reflita a leitura convencional da esquerda para a direita (do menor para o maior número de grupos).
- A visualização desses gráficos permite validar a escolha do número de clusters (como o valor 8 utilizado nos blocos anteriores). O ponto ideal é geralmente encontrado onde a curva do Fator Mojena apresenta um "cotovelo" ou estabilização, cruzando os patamares recomendados pela literatura (entre 1.25 e 1.50).

Método de Ward

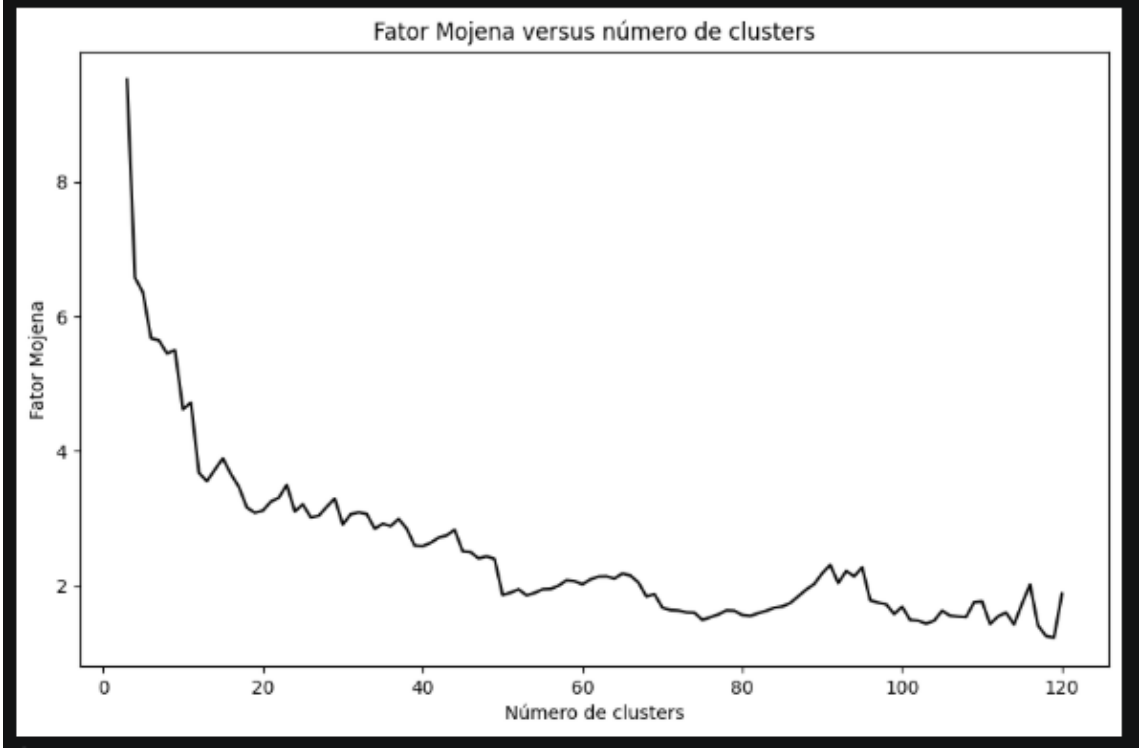
```python
moj_factor = []
N = len(Z_ward)  # Número total de fusões (linhas na matriz Z_ward)

N_clusters_values = list(range(3, N))
for i in N_clusters_values:
    aux = Z_ward[0:i, :]
    moj_factor.append(mojena_fator(aux))

# Reverse both the moj_factor list and the list of cluster numbers for plotting
moj_factor.reverse()

plt.figure(figsize=(10, 6))
plt.plot(N_clusters_values, moj_factor, color='black')
plt.title('Fator Mojena versus número de clusters')
plt.xlabel('Número de clusters')
plt.ylabel('Fator Mojena')
plt.show()
```

✓ 0.0s



O gráfico apresenta uma queda mais suave e estruturada. O fator de Mojena estabiliza próximo a valores baixos conforme o número de clusters aumenta, indicando onde as fusões param de gerar "saltos" significativos na variância interna.

Método Single

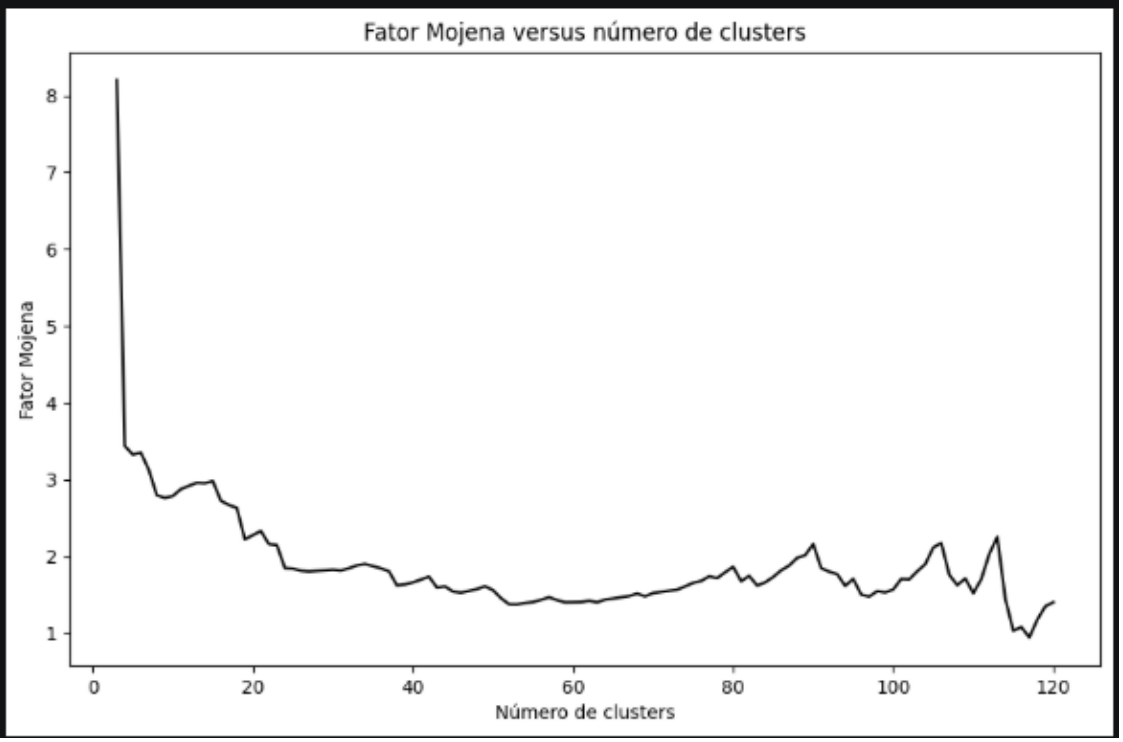
```python
moj_factor = []
N = len(Z_single)  # Número total de fusões (linhas na matriz Z_single)

N_clusters_values = list(range(3, N))
for i in N_clusters_values:
    aux = Z_single[0:i, :]
    moj_factor.append(mojena_fator(aux))

# Reverse both the moj_factor list and the list of cluster numbers for plotting
moj_factor.reverse()

plt.figure(figsize=(10, 6))
plt.plot(N_clusters_values, moj_factor, color='black')
plt.title('Fator Mojena versus número de clusters')
plt.xlabel('Número de clusters')
plt.ylabel('Fator Mojena')
plt.show()
```

✓ 0.0s



Observa-se uma queda abrupta inicial. Devido ao efeito de encadeamento, o fator permanece baixo e estável por um longo intervalo, confirmando que a adição de novos pontos individuais não altera drasticamente a estrutura de distância mínima do modelo.

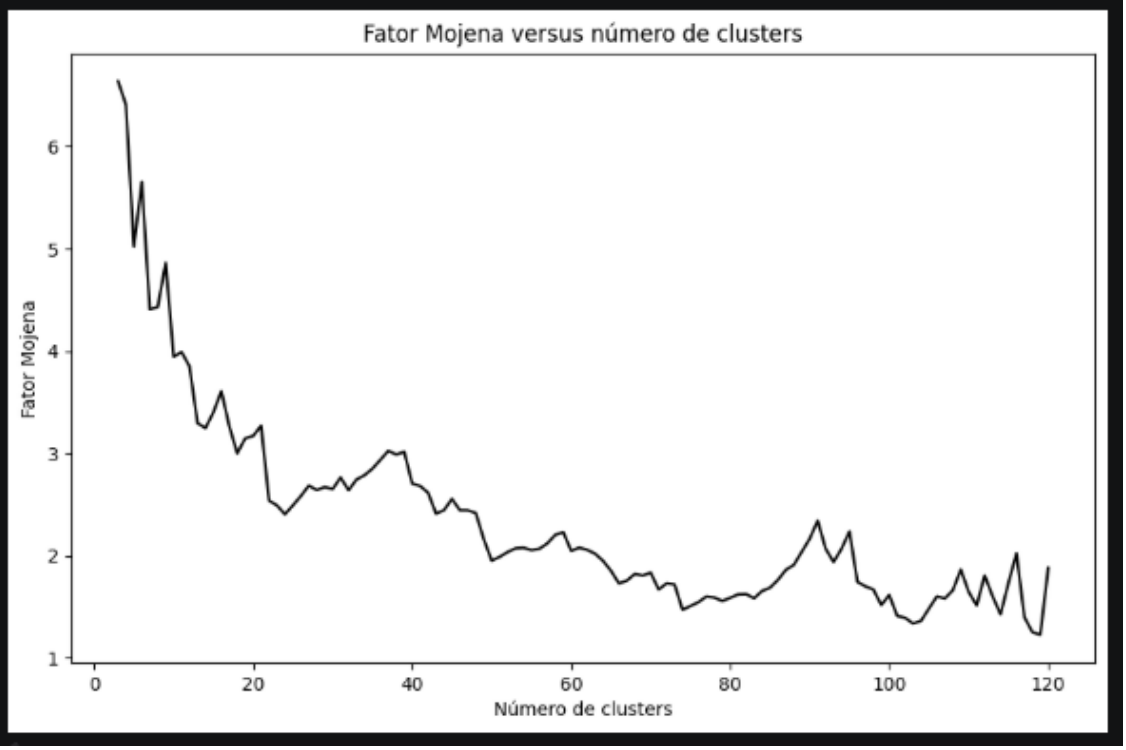
Método Complete

```
moj_factor = []
N = len(Z_complete)  # Número total de fusões (linhas na matriz Z_complete)

N_clusters_values = list(range(3, N))
for i in N_clusters_values:
    aux = Z_complete[0:i, :]
    moj_factor.append(mojena_fator(aux))

# Reverse both the moj_factor list and the list of cluster numbers for plotting
moj_factor.reverse()

plt.figure(figsize=(10, 6))
plt.plot(N_clusters_values, moj_factor, color='black')
plt.title('Fator Mojena versus número de clusters')
plt.xlabel('Número de clusters')
plt.ylabel('Fator Mojena')
plt.show()
✓ 0.0s
```



Exibe uma oscilação moderada. Por focar na distância máxima, o fator de Mojena ajuda a identificar os pontos de corte onde a expansão do "diâmetro" do cluster se torna estatisticamente injustificável.

8: Automação do Ponto de Corte: Regra de Parada de Mojena

```python
# Função para encontrar o melhor número de clusters baseado no fator Mojena

def mojena_stopping_rule(Z, k_factor):
    """
    Implementa o método de Mojena para agrupamento hierárquico.
    Z: Matriz linkage do scipy
    k_factor: Valor padrão geralmente entre 1.25 e 1.50
    """
    # Distâncias de fusão (terceira coluna da matriz Z)
    distances = Z[:, 2]

    # Calcular média e desvio padrão das distâncias
    mean_dist = np.mean(distances)
    std_dist = np.std(distances)

    # Calcular critério de Mojena (k) para cada estágio
    # k_j = (alpha_j - mean) / std
    mojena_k = (distances - mean_dist) / std_dist

    # Encontrar o estágio onde a distância é "significativamente" maior
    # O corte é geralmente onde mojena_k > k_factor
    # Pegamos o último estágio que está abaixo do limite para cortar

    stopping_stage = np.argmax(mojena_k > k_factor) - 1

    # Número de clusters = Total de elementos - estágio de parada
    num_clusters = len(distances) - stopping_stage

    return num_clusters, mojena_k
```

Este bloco implementa a função mojena_stopping_rule, que transforma o cálculo estatístico em uma decisão objetiva sobre o número de clusters a serem utilizados:

- **Critério de Parada:** A função identifica o primeiro estágio do agrupamento onde a distância de fusão é significativamente maior que a média histórica. Esse "salto" é definido quando o valor de k supera um limite pré-estabelecido (k_factor, geralmente entre 1.25 e 1.50).
- **Cálculo do Estágio de Parada:** Através do comando “np.argmax(mojena_k > k_factor) – 1”, o algoritmo localiza o último estágio de agrupamento que ainda é considerado estatisticamente coeso antes de ocorrer uma união de grupos muito dissimilares.
- **Determinação Automática (num_clusters):** A função subtrai o estágio de parada do número total de elementos da amostra. O resultado é a quantidade exata de clusters que o modelo deve adotar para garantir a melhor segmentação possível.
- **Vantagem Metodológica:** Diferente da análise puramente visual do dendrograma, esta função permite que o pesquisador justifique o número de clusters com base na distribuição estatística das distâncias, conferindo maior rigor científico ao modelo de agrupamento.

9: Determinação Estatística e Visual dos Clusters

```
n_clusters, k_values = mojena_stopping_rule(Z_ward, k_factor=1.25)
print(f"Número ideal de clusters (Mojena): {n_clusters}")
✓ 0.0s

Número ideal de clusters (Mojena): 8
```

```
n_clusters, k_values = mojena_stopping_rule(Z_single, k_factor=1.25)
print(f"Número ideal de clusters (Mojena): {n_clusters}")
✓ 0.0s

Número ideal de clusters (Mojena): 6
```

```
n_clusters, k_values = mojena_stopping_rule(Z_complete, k_factor=1.25)
print(f"Número ideal de clusters (Mojena): {n_clusters}")
✓ 0.0s

Número ideal de clusters (Mojena): 9
```

Este bloco consolida a transição entre o cálculo matemático do Método de Mojena e a aplicação prática do algoritmo AgglomerativeClustering, comparando como cada critério de ligação traduz os dados no espaço bidimensional (Qualidade de Vida vs. Corrupção). Foi escolhido resultados do critério para o $k = 1.25$

- **Método de Ward:** Identificou 8 clusters como o ideal. É o método que gera a segmentação mais equilibrada, fatiando a nuvem de dados em blocos compactos e facilitando a identificação de perfis socioeconômicos distintos.
- **Método Single:** Sugeriu 6 clusters. Reflete a tendência do método de "ignorar" pequenas variações, criando um supergrupo central e separando apenas os casos mais extremos (outliers).
- **Método Complete:** Apontou para 9 clusters. Por focar no diâmetro máximo dos grupos, ele tende a ser mais conservador, dividindo a amostra em mais partes para garantir que nenhum membro fuja do limite de similaridade.

Discussão e Considerações Finais

A análise comparativa demonstrou que a escolha do método de ligação (linkage) é o fator mais crítico na definição da estrutura dos grupos. O **Método de Ward** provou ser o mais eficaz para a segmentação de perfis nacionais, gerando clusters equilibrados e com alta coesão interna. Esta técnica facilitou a distinção clara entre blocos de países com diferentes níveis de bem-estar e integridade, permitindo identificar padrões socioeconômicos que fazem sentido tanto estatística quanto geograficamente, validando a eficácia da normalização Min-Max realizada na etapa de preparação.

Em contrapartida, o **Método Single** apresentou o clássico efeito de encadeamento, onde a maioria das observações é absorvida por um único grupo dominante. Embora seja limitado para criar uma classificação geral útil, este método destacou-se pela sua capacidade de isolar anomalias estatísticas (outliers). Graças à identificação nominal nos gráficos de dispersão, foi possível visualizar que países como Singapura e Ruanda foram rapidamente separados do restante da amostra, evidenciando correlações entre felicidade e corrupção que fogem totalmente à curva de tendência global.

O **Método Complete** ofereceu uma perspectiva intermediária e mais conservadora. Ao focar na distância máxima entre os membros, ele evitou o encadeamento do método Single, mas resultou em uma fragmentação maior do que a observada no método de Ward. Esta abordagem revelou-se útil para garantir que nenhum país dentro de um mesmo cluster apresentasse uma disparidade extrema em relação aos demais, sendo ideal para cenários onde a similaridade absoluta entre todos os membros é uma prioridade.

Por fim, a implementação da **Regra de Parada de Mojena** foi fundamental para conferir rigor metodológico ao estudo. Ao substituir a análise visual subjetiva dos dendrogramas por uma decisão baseada em saltos estatísticos de variância, foi possível justificar o número de grupos (como os **8 clusters**) de forma científica. Conclui-se que o fluxo de trabalho adotado — da preparação rigorosa à automação do ponto de corte — permite transformar dados brutos em uma ferramenta robusta para a compreensão das complexas disparidades globais nos indicadores de desenvolvimento humano.